Lab 8: Build Resilient File Processing with Try Catch

Topic: 4

Objective: LO4 - Handle business and system exceptions with controlled recovery

Alignment: A5, A7, A8 | K8, K13

Suggested time: 70 minutes

Scenario and outcome

Process a file manifest, classify missing or invalid files, retry transient access failures and produce a clean exception summary without exposing sensitive data.

You will produce: A Try Catch workflow with bounded retry, Finally cleanup and a structured exception report

Folder contents

- Main.xaml - editable UiPath Studio starter workflow.
- project.json - UiPath project manifest.
- mock-build-resilient-file-processing-with-try-catch.csv - portable mock source data.
- mock-build-resilient-file-processing-with-try-catch.xlsx - formatted mock workbook with Data and Rules sheets.
- CHECKLIST.md - completion and evidence checklist.

Preparation

1. Use UiPath Studio Community 2024.10 or later.
2. Extract or clone this entire folder. Keep the data files beside project.json.
3. Open project.json in Studio and allow package restore. Upgrade dependency versions if Studio requests it.
4. Use mock data only. Do not enter passwords, tokens or real personal data.

Procedure

1. Open the manifest and define which outcomes are business exceptions versus system exceptions.
2. Open Main.xaml. Inside the For Each Row, place normal file processing in a Try block.
3. If Exists is No or ContentValid is No, throw or create a BusinessRuleException with TransactionID and a safe reason.
4. For Locked = Yes, use Retry Scope or a bounded loop with at most three attempts and a short delay. Log each attempt.
5. Catch BusinessRuleException separately from System.Exception. Route business exceptions to Review; route unresolved system failures to Technical Support.
6. In Finally, release any file/application resource and append a transaction result regardless of outcome.
7. Write TransactionID, Outcome, Attempts and SafeMessage to output/exception-summary.csv. Do not write passwords, tokens or full confidential content.
8. Run all six manifest rows and compare outcome with ExpectedOutcome. Explain why TX6006 must not be reported as successful.

9. Capture the Try Catch design and the final exception summary.

Expected result

All six rows produce one final result, retries never exceed three, business and system exceptions are distinct, and no sensitive value appears in logs.

Troubleshooting

If the workflow continues after a critical system failure, check ContinueOnError and the Catch recovery path. If results duplicate, append only once in Finally.

Evidence and acceptance criteria

output/exception-summary.csv and screenshots of Try Catch plus one retry log sequence.

The submission is acceptable when the expected result is visible, output files reconcile to the mock input, and the evidence identifies the tested transaction or process.